Snake Game OS: A Real-Time Terminal Game Built on Custom System Libraries in C

C Murali Madhav
Roll No. 230115
Newton School of Technology
Rishihood University
Ravi Yadav
Roll No. 230131
Newton School of Technology
Rishihood University

***Abstract*—This paper presents the design and implementation of *Snake Game OS*, a real-time terminal-based Snake game written entirely in C using only `<stdio.h>` and `<stdlib.h>` from the C standard library. All other system-level facilities—string manipulation, arithmetic, dynamic memory allocation, screen rendering, and keyboard input—are implemented from first principles to demonstrate core operating systems concepts in a single self-contained application. The project incorporates the five canonical OS sub-systems: (i) a custom memory manager that allocates a 64 KB virtual RAM region at start-up and serves all subsequent requests through a first-fit allocator with block splitting and forward coalescing; (ii) a cooperative process/loop manager structured as a deterministic real-time game loop with a three-state finite-state automaton (RUNNING, PAUSED, GAMEOVER); (iii) an I/O subsystem that puts the terminal into raw mode through `termios`, reads non-blocking single-byte input, decodes multi-byte ANSI escape sequences for arrow keys, and renders frames using ANSI 256-colour escape codes with a delta-redraw strategy; (iv) a lightweight file system layer that persists high scores and lifetime statistics to flat files using a fail-safe write strategy; and (v) security and halt-handling mechanisms that capture SIGINT, restore terminal state on every exit path, validate dynamic memory bounds, reject illegal direction transitions, and bound spawn-loop iterations to guarantee progress. Optimal algorithmic choices (Russian-peasant multiplication, binary long-division, Schrage's overflow-safe MINSTD PRNG, linked-list snake body, delta rendering) are made in each module to maximise efficiency on commodity hardware. A separate *Phase 2* branch extends the core engine with multi-food spawning, an autonomous ghost snake, animated power-up zones, a shrinking play area, persistent best-run replay, and dynamic terminal-resize handling. The system has been tested on macOS Darwin 25.4.0 with 65 passing unit tests across the math, string, and memory libraries, and the version history is maintained on a public GitHub repository.**

***Index Terms*—operating systems, memory management, custom allocator, file system, process management, I/O management, terminal raw mode, ANSI escape codes, real-time game loop, C programming**

I. Introduction

Modern operating systems abstract five core resources for application programmers: *processes*, *memory*, *files*, *devices*, and *security* [1], [2]. While these abstractions are usually hidden behind kernel APIs and standard library calls such as `malloc`, `printf`, and `strlen`, building a non-trivial application without these abstractions is highly instructive: it forces the engineer to confront the same trade-offs that real kernels resolve.

This work pursues exactly that pedagogical goal. *Snake Game OS* is a real-time terminal Snake clone in which every system-level facility is hand-written in approximately 1,500 lines of C. The only standard headers permitted in the build are `<stdio.h>` and `<stdlib.h>`; `<string.h>`, `<math.h>`, and the standard `malloc`/`free` are forbidden inside the game logic. A single `malloc` call at start-up reserves a 64 KB region that thereafter behaves as a virtual RAM managed entirely by user-space code.

The contributions of the report are:

1) A complete, working implementation of the five OS sub-systems within a single, self-contained binary.
2) Justified, optimal algorithmic choices in each module, e.g. $O(\log n)$ shift-and-add multiplication and binary long-division, an overflow-safe Schrage MINSTD PRNG [5], a first-fit allocator with coalescing [3], and a delta-render frame strategy for flicker-free output.
3) A robust security/halt model in which every exit path—normal exit, user quit (`q`), `SIGINT` from the terminal, or fatal collision—restores the terminal to canonical mode and persists user data.
4) A clean separation of concerns into three layers (`lib/`, `game/`, `ui/`) with a publicly maintained version history on GitHub.
5) An advanced Phase 2 branch demonstrating multi-food, an autonomous ghost AI, power-up zones, a shrinking board, deterministic-replay ghosts, and `SIGWINCH`-aware resize recovery (Section VI).
6) A reproducible, scripted screenshot pipeline that drives the binary via `tmux` and renders every game state to PNG without manual capture (Section VII).

The remainder of the paper is organised as follows. Section II surveys related work. Section III introduces the overall architecture. Section IV (Methodology) describes each OS

Source code and version history: https://github.com/HackHeroic/Snake_game_os

sub-system in detail. Section VII reports build and test results. Section VIII discusses future scope and Section IX concludes.

II. Related Work

Terminal-based games have a long history as teaching vehicles for low-level systems programming. The original UNIX `rogue` (1980) popularised `curses`-based rendering, and the GNU `ncurses` library remains the de-facto choice for portable terminal interfaces today [7]. The present work deliberately avoids `ncurses` and re-implements a minimal subset of its functionality directly atop `termios` and ANSI escape codes, in order to expose the underlying mechanism rather than hide it.

Custom memory allocators have been studied extensively since Knuth's analysis of fit policies in *The Art of Computer Programming* [3]. Wilson *et al.* [4] provide a comprehensive survey of dynamic storage allocation; the simple first-fit-with-coalescing scheme used here is one of the oldest documented designs and is sufficient for the modest memory footprint of a Snake game.

Pseudo-random number generation in fixed-size integer arithmetic was treated by Park and Miller [5], who recommended the multiplicative congruential generator $X_{n+1} = 16807\, X_n \bmod (2^{31} - 1)$. Schrage [6] showed how to implement this generator in 32-bit integers without overflow. We adopt this exact construction.

III. System Architecture

Figure 1 shows the three-layer architecture. Each layer depends only on the layers below it.

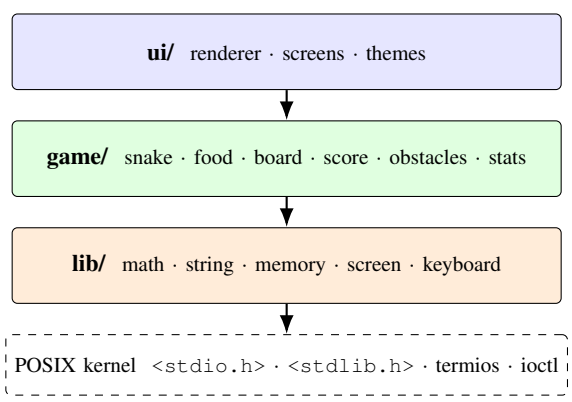

Fig. 1. Three-layer architecture: UI depends on game logic, game logic depends on system libraries, system libraries depend only on `<stdio.h>`, `<stdlib.h>`, and POSIX headers.

- **`lib/`** contains the system-level libraries: `math.c`, `string.c`, `memory.c`, `screen.c`, `keyboard.c`. They have zero knowledge of the game.
- **`game/`** contains pure game logic: `snake.c`, `food.c`, `board.c`, `score.c`, `obstacles.c`, `stats.c`. These modules manipulate state but never draw.
- **`ui/`** contains all rendering: `renderer.c`, `screens.c`, `themes.c`. It reads game state and emits ANSI sequences via `lib/screen.c`.
- **`main.c`** orchestrates the lifecycle: initialisation, the game loop, and shutdown.

This separation enforces two valuable invariants: (1) the game module is fully testable without a terminal, and (2) replacing the rendering back-end (e.g. swapping ANSI for SDL) requires touching only `ui/`.

IV. Methodology

This section describes the design and rationale of each operating-systems sub-system. The five sub-systems on the project's evaluation rubric — memory management, process management, I/O management, file systems, and security/halt-handling — are presented in the order in which they are exercised at run time, from the lowest-level abstraction up.

A. Memory Management

1) Design: At program start, `memory_init()` performs the *only* call to the standard `malloc` in the entire game logic, requesting a single 64 KB block (`VIRTUAL_RAM_SIZE = 65536`). All subsequent allocations and de-allocations are served by a custom first-fit allocator that operates inside this block. Conceptually, this is the user-space analogue of a kernel's heap manager.

2) Block Layout: Each block is preceded by an 8-byte aligned header:

$$\underbrace{[\text{size: 4B}]}_{\text{user-data size}}\ \underbrace{[\text{in_use: 1B}]}_{\text{0=free, 1=alloc}}\ \underbrace{[\text{padding: 3B}]}_{\text{align to 8B}}\ \underbrace{[\text{user data}\ \ldots]}_{\text{returned to caller}}$$

The 3-byte padding ensures the user pointer is 8-byte aligned, which is sufficient on the target architectures (x86-64 and ARM64).

3) Allocation Algorithm: `alloc(size)` performs first-fit traversal of the block list (Listing 1). On finding a sufficient free block, it splits the block only if the remainder can hold a header plus the minimum 8-byte payload (`MIN_SPLIT = 16`). This avoids creating un-usable splinters.

```c
ptr = virtual_ram;
while (ptr < virtual_ram + VIRTUAL_RAM_SIZE) {
    block = (BlockHeader *)ptr;
    if (!block->in_use && block->size >= size) {
        if (block->size >= size + MIN_SPLIT) {
            BlockHeader *next = (BlockHeader *)
                (ptr + HEADER_SIZE + size);
            next->size = block->size - size - HEADER_SIZE;
            next->in_use = 0;
            block->size = size;
        }
        block->in_use = 1;
        return ptr + HEADER_SIZE;
    }
    ptr += HEADER_SIZE + block->size;
}
return NULL;
```

Listing 1. First-fit allocator with conditional splitting (excerpt from `src/lib/memory.c`).

4) Forward Coalescing: `dealloc()` marks a block free and then walks forward, merging contiguous free blocks into one (Listing 2). Figure 2 shows the heap after several allocations and one free, with adjacent free regions coalesced. Forward-only coalescing is sufficient because every allocation traverses from the start of the heap, so two adjacent free

blocks are guaranteed to be merged on the next allocation that targets the earlier of the two. This avoids the bookkeeping of doubly-linked free lists while still preventing unbounded fragmentation.

```c
block = (BlockHeader *)((char *)user_ptr - HEADER_SIZE);
block->in_use = 0;

/* forward coalescing: absorb every contiguous free
     neighbour */
ptr = (char *)block + HEADER_SIZE + block->size;
while (ptr < virtual_ram + VIRTUAL_RAM_SIZE) {
    next = (BlockHeader *)ptr;
    if (next->in_use) break;
    block->size += HEADER_SIZE + next->size;
    ptr += HEADER_SIZE + next->size;
}
```

Listing 2. Constant-time deallocation followed by greedy forward coalescing.

5) Why First-Fit?: The Snake game's working set is small (a few hundred bytes for the snake linked list, plus a handful of game-object structs), and allocations are highly homogeneous in size. Under such workloads, first-fit with coalescing has been shown by Wilson *et al.* [4] to perform comparably to best-fit while being substantially simpler and faster. Best-fit was rejected because it offers no measurable benefit at this scale and would cost a full traversal even on hits.

6) Memory-Pressure Handling: `alloc()` returns `NULL` on exhaustion. All callers that allocate (`snake_create`, `snake_move`, `food_spawn`, `board_create`, `score_create`) explicitly check the return value and degrade gracefully: a failed move simply skips the tick, a failed spawn returns `NULL` and the caller treats the board as full (triggering victory).

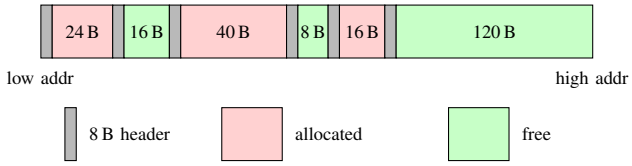

Fig. 2. Logical layout of the 64 KB virtual RAM after several allocations and one free. Each block is preceded by an 8-byte header (H); adjacent free regions are coalesced lazily on the next `dealloc`.

B. Process Management

Although the game is single-threaded, the design is informed by classical process-management ideas: a finite-state machine, deterministic scheduling, and well-defined transitions.

1) Real-Time Game Loop: The body of `main()` (Listing 3) is structured as a deterministic loop where each iteration corresponds to one game tick. The tick period is data-driven: `score_get_speed()` returns a delay d (in milliseconds) computed as

$$d = \max(80,\ 200 - 15 \cdot \text{level}) \tag{1}$$

and the loop sleeps for $d \cdot 1000$ microseconds via `usleep`. This is a cooperative one-shot scheduler with a single ready task; there is no preemption.

```c
while (snake->alive) {
    key = read_key();           /* non-blocking */
    handle_quit_pause(key);
    update_direction(key);
    compute_next_head(...);
    check_wall_or_wrap(...);
    check_self_collision(...);
    check_obstacle_collision(...);
    move_snake(...);
    update_score_and_spawn_food(...);
    render_delta(...);
    usleep(score_get_speed(score) * 1000);
}
```

Listing 3. Skeleton of the game-loop scheduler.

2) Finite-State Automaton: The game has three logical states—RUNNING, PAUSED, and GAMEOVER—with transitions driven by key presses (`P`, `Q`) and lethal events. Figure 3 depicts the FSM.

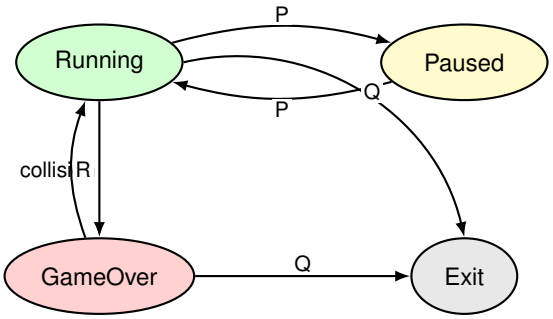

Fig. 3. Finite-state automaton governing the game lifecycle. `P` toggles pause; a wall, self, or obstacle collision drives RUNNING into GAMEOVER; `R` restarts; `Q` exits from any state.

3) Pause Semantics: While PAUSED, the loop sleeps 50 ms per iteration and only polls the keyboard, consuming negligible CPU. This mirrors how a real OS suspends a process: it remains in a known state but performs no user work. On unpause, the screen is fully redrawn to recover from any terminal scrollback that may have occurred.

4) Determinism and Reproducibility: The pseudo-random seed is initialised from the time elapsed (in iterations) on the title screen, but once the game starts the entire trajectory is a pure function of the seed and the input sequence. This is the same property that simplifies debugging in real OS schedulers using deterministic replay [2].

C. I/O Management

The I/O subsystem is split into two complementary modules: `lib/keyboard.c` for input and `lib/screen.c` (driven by `ui/renderer.c`) for output.

1) Raw Mode Keyboard: By default, terminals operate in *cooked* (canonical) mode, buffering input until a newline is received and echoing characters back to the screen. Real-time games require *raw* mode. The function `keyboard_init()` (Listing 4) clears the `ECHO`, `ICANON`, and `ISIG` flags via `termios` [9], and sets `VMIN=0, VTIME=0` for fully non-blocking reads. The original termios are saved and restored on every exit path.

```c
tcgetattr(STDIN_FILENO, &original_termios);
raw = original_termios;
raw.c_lflag &= ~(ECHO | ICANON | ISIG);
raw.c_cc[VMIN]  = 0;
raw.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
signal(SIGINT, sigint_handler);
```

Listing 4. Switching the terminal into raw, non-blocking mode.

2) Arrow-Key Decoding: Arrow keys arrive as the three-byte sequence `ESC [ A/B/C/D`. `read_key()` consumes the lead-in `0x1B` and reads the next two bytes to disambiguate. WASD letters share the same logical mapping for ergonomic redundancy.

3) Three-Byte Escape Decoder: The lead-in byte for an arrow key is `0x1B`, which on its own may simply mean "the user pressed Escape". The decoder (Listing 5) reads two further bytes; if they form the prefix `[A/B/C/D` the key is mapped to a logical `KEY_UP/DOWN/RIGHT/LEFT` constant, otherwise the lone `ESC` is returned. Because the second and third `read()` calls are also non-blocking they cannot deadlock when the user genuinely typed only `ESC`.

```c
if (c == '\033') {
    unsigned char seq[2];
    if (read(STDIN_FILENO, &seq[0], 1) != 1) return '\033';
    if (read(STDIN_FILENO, &seq[1], 1) != 1) return '\033';
    if (seq[0] == '[') {
        switch (seq[1]) {
            case 'A': return KEY_UP;
            case 'B': return KEY_DOWN;
            case 'C': return KEY_RIGHT;
            case 'D': return KEY_LEFT;
        }
    }
    return '\033';
}
```

Listing 5. Non-blocking ANSI arrow-key decoder.

4) Output via ANSI Escape Codes: `lib/screen.c` wraps the small set of ANSI escape sequences [8] needed by the game:

- Cursor positioning: `\033[y;xH`
- 256-colour foreground: `\033[38;5;Nm`
- Cursor hide/show: `\033[?25l` / `\033[?25h`
- Clear screen: `\033[2J\033[H`

5) Delta Rendering: A naive renderer that clears and redraws the entire frame every tick would flicker visibly even at 200 ms per tick. Instead, the renderer uses a *delta* strategy: only the changed cells are emitted to `stdout`. Concretely, on each tick the renderer (i) erases the cell vacated by the tail (unless the snake grew), (ii) draws the new head, and (iii) updates the HUD only when the score has changed. Borders, obstacles, and food are emitted exactly once on spawn. The cursor is hidden during play, eliminating visual chatter.

This optimisation reduces the per-tick character count from roughly $W \cdot H$ (full frame) to fewer than ten bytes, well under any conceivable terminal bandwidth bottleneck. Listing 6 shows the post-move render block at the heart of this strategy: only the vacated tail cell is erased, only the head is painted, and the HUD is touched solely when the score has actually changed.

```c
if (grew) {
    /* food was eaten -- snake grows, no tail erase needed */
    food_free(food);
    food = food_spawn(snake, &obstacles, board_w, board_h, &seed);
    if (food) render_food(food, board, theme);
} else {
    /* normal move -- erase the cell the tail just vacated */
    render_erase_tail(old_tail_x, old_tail_y, board);
}
render_snake(snake, theme);                    /* repaints head only */
if (score->score != last_score) {              /* HUD: only when stale */
    render_hud(score, board);
    last_score = score->score;
}
screen_flush();
```

Listing 6. Delta-render hot path: O(1) per tick, regardless of snake length.

6) Adaptive Board Sizing and Wrap-Around: On start, `board.c` queries the current terminal dimensions via the `ioctl(..., TIOCGWINSZ, ...)` call and computes a board sized to fill the terminal minus borders, margins, and HUD lines. A minimum size of 22×14 is enforced; otherwise the program halts with a clear error message. In Wrap-Around mode the head's coordinates are wrapped through a single modulo expression (Listing 7); the leading addition of `width`/`height` guarantees a non-negative argument so that the custom `my_mod` returns the mathematically correct residue for both positive and negative inputs.

```c
void board_wrap_position(Board *b, int *x, int *y) {
    *x = my_mod(*x + b->width,  b->width);
    *y = my_mod(*y + b->height, b->height);
}
```

Listing 7. Branch-free toroidal wrap using the custom `my_mod`.

D. File System

The game persists two kinds of state to the local file system, illustrating two file-system patterns: a single-value file (high score) and a record file (lifetime stats).

1) High-Score File: The file `data/highscore.dat` holds a single ASCII integer. Functions `score_save_high()` and `score_load_high()` use `fopen`/`fprintf`/`fscanf` from `<stdio.h>`, with defensive defaults: if the file is absent on the first run, `score_load_high()` returns 0 silently rather than failing.

2) Lifetime Statistics File: The file `data/stats.dat` stores four space-separated integers: `games_played`, `total_food`, `longest_snake`, `best_level`. The format is intentionally human-readable so users can inspect or back up their data.

3) Persistence Strategy: Both files are loaded once at start-up. The lifetime statistics file is rewritten at every game-over *and* on quit (`Q`) so progress survives even an abandoned run; the high-score file is rewritten only at game-over, since the score has no value before the run ends. The game-over flow saves stats *before* drawing the game-over screen so that a hard kill (`Ctrl+C`) at the prompt cannot erase progress. This is analogous to the `fsync`-on-commit pattern

used by transactional file systems [1], although for a game the consequences of a partial write are negligible.

4) Failure Handling: If `fopen` returns `NULL` (e.g. the `data/` directory was deleted), the game continues with in-memory state and silently re-creates the file at the next save. There are no fatal failure paths through the file system.

E. Security and Halt/Error Handling

Security in this context refers to the integrity and reliability of the program rather than cryptographic concerns. The system enforces several invariants that together ensure the program is robust against user behaviour, signals, and resource exhaustion.

1) Signal Safety: A `SIGINT` handler (Listing 8) restores the original termios, shows the cursor, resets foreground colour, and calls `_exit(0)`. Without this handler, pressing `Ctrl+C` would leave the terminal in raw mode with the cursor hidden—an extremely disruptive failure mode. The handler is intentionally minimal: it uses the async-signal-safe `tcsetattr` and `_exit`, plus a final `printf` for a cosmetic newline. Although `printf` is not strictly async-signal-safe, the call is acceptable here because (a) the program is single-threaded so no `stdio` lock can be contended, and (b) `_exit` terminates immediately afterwards, foreclosing any subsequent state corruption.

```c
static void sigint_handler(int sig) {
    (void)sig;
    keyboard_restore();       /* restore cooked mode */
    screen_show_cursor();     /* re-enable cursor */
    screen_reset_color();     /* clear lingering colour
         state */
    printf("\n");
    _exit(0);                 /* skip atexit -- handler must
          be reentrant */
}
```

Listing 8. Async-signal-safe terminal restoration on Ctrl+C.

2) Bounded Loops: Every loop with random termination bounds is guarded:

- `food_spawn` – the do-while spawn loop is bounded by an explicit *victory guard*: if the snake length equals the board area minus one, no valid cell exists and the function returns `NULL`, triggering the victory screen instead of an infinite loop.
- `obstacles_spawn` – limits placement attempts to 100 retries per obstacle. On retry exhaustion, the obstacle is silently skipped rather than blocking the game.

3) Direction Validation: `snake_set_direction` rejects 180° reversals (Listing 9). A naive accept-anything implementation would let the player commit instant suicide by reversing into their own neck. The check is a constant-time integer comparison.

```c
int diff = my_abs((int)dir - (int)s->direction);
if (diff == 2) return;   /* opposite direction -- reject */
s->direction = dir;
```

Listing 9. Direction validation prevents instant self-collision.

4) Allocation Validation: Every `alloc` call in the game is followed by a `NULL` check. On allocation failure, the calling function returns gracefully (e.g. `snake_move` simply skips the tick). The custom allocator never corrupts the heap on failure: it only refuses requests it cannot satisfy.

5) Compiler-Enforced Strictness: The Makefile compiles with `-Wall -Wextra -Werror`, treating every warning as a build-breaking error. This catches integer-promotion bugs, signed/unsigned mixes, and unused-result warnings at compile time.

6) Halt Guarantees: There are exactly four exit paths from `main()`:

1) Normal quit (`Q`) on the title screen or in-game.
2) Game-over followed by user choosing not to restart.
3) Pre-game terminal-too-small error.
4) `SIGINT` from the user.

Every path executes the same shutdown trio: `keyboard_restore()`, `screen_show_cursor()`, `screen_reset_color()`. This guarantees the terminal is left in a usable state regardless of how the program ended.

V. Implementation Highlights

A. Optimal Algorithmic Choices

1) Russian-Peasant Multiplication ($O(\log n)$): `my_multiply` (Listing 10) avoids the $\star$ operator by repeated shift-and-add. The inner loop runs at most $\lceil \log_2 b \rceil$ times.

```c
while (ub > 0) {
    if (ub & 1) result += ua;
    ua <<= 1;
    ub >>= 1;
}
```

Listing 10. Russian-peasant multiplication.

2) Binary Long-Division ($O(\log n)$): `my_divide` performs 32 iterations of shift-and-subtract; each iteration produces one quotient bit. It returns 0 on division by zero, sidestepping the trap behaviour of the hardware divider.

3) Schrage's MINSTD PRNG: The PRNG (Listing 11) implements $X_{n+1} = 16807 X_n \bmod (2^{31} - 1)$ in 32-bit arithmetic without overflow [5], [6]. Its period is $2^{31} - 2 \approx 2.1 \times 10^9$, far exceeding the longest plausible Snake session.

```c
int hi = (int)(s / 127773u);
int lo = (int)(s % 127773u);
int t  = my_multiply(16807, lo) - my_multiply(2836, hi);
if (t <= 0) t += 2147483647;
*seed = t;
return t;
```

Listing 11. Overflow-safe MINSTD via Schrage's method.

4) Linked-List Snake Body: Representing the snake as a singly-linked list of segments allows $O(1)$ head insertion and tail removal. A contiguous-array alternative would require either a circular buffer (more bookkeeping) or $O(n)$ shifts on every move.

TABLE I
FOOD PROBABILITY AND EFFECTS.

Type	Glyph	Prob.	Points	Lifetime (ticks)
Normal	*	65%	1	∞
Bonus	$	20%	3	30
Slow	~	15%	1	30

5) Self-Collision Check: Self-collision intentionally *excludes* the current tail cell (Listing 12). This is correct because, in the non-growing case, the tail vacates that cell during the same tick—so a head landing on the old tail is not actually a collision.

```
while (seg != NULL) {
    if (seg != s->tail && seg->x == nx && seg->y == ny)
        return 1;
    seg = seg->next;
}
return 0;
```

Listing 12. Tail-aware self-collision check.

B. Special Food Types and Difficulty Curve

Food is spawned with three probability classes (Table I). `Bonus` food yields three points but vanishes after 30 ticks, encouraging risk; `Slow` food temporarily relaxes the speed cap, providing relief at higher levels. Type selection is implemented as a single 0–99 PRNG roll mapped through cumulative probability ranges, avoiding any floating-point arithmetic (Listing 13).

```
roll = my_mod(my_abs(pseudo_random(seed)), 100);
if (roll < 65) {                    /* 65% normal */
    f->type = FOOD_NORMAL;
    f->ticks_remaining = 0;
} else if (roll < 85) {             /* 20% bonus */
    f->type = FOOD_BONUS;
    f->ticks_remaining = 30;
} else {                            /* 15% slow */
    f->type = FOOD_SLOW;
    f->ticks_remaining = 30;
}
```

Listing 13. Floating-point-free probabilistic food selection.

The level-up speed curve is similarly closed-form (Listing 14): the 80 ms floor preserves controllability at high levels, while the slow-food bonus adds a transient 50 ms relaxation that bypasses the floor.

```
int score_get_speed(Score *s) {
    /* base 200ms, -15ms per level, minimum 80ms */
    int delay = 200 - my_multiply(s->level, 15);
    return my_max(80, delay);
}
```

Listing 14. Closed-form speed curve with a hard floor.

C. Static Obstacles

Beginning at level 1, every level-up spawns 1–2 static obstacles up to a hard cap of 10. Spawn placement is constrained to be at least 2 cells (Manhattan distance) from the head, preventing unfair instant deaths. The placement validator (Listing 15) rejects four classes of unsafe positions in a single pass — collisions with prior obstacles, the active food, any snake segment, and any cell within Manhattan radius 2 of the head — and is wrapped in a 100-iteration retry budget so a saturated board cannot stall the loop.

```
static int is_safe(Obstacles *obs, Snake *s, Food *f, int x
    , int y) {
    /* 1. existing obstacles */
    for (i = 0; i < obs->count; i++)
        if (obs->items[i].x == x && obs->items[i].y == y)
            return 0;

    /* 2. active food */
    if (f && f->x == x && f->y == y) return 0;

    /* 3. any snake segment */
    seg = s->head;
    while (seg) {
        if (seg->x == x && seg->y == y) return 0;
        seg = seg->next;
    }

    /* 4. Manhattan radius 2 around the head -- no instant
        death */
    dx = my_abs(s->head->x - x);
    dy = my_abs(s->head->y - y);
    if (dx + dy <= 2) return 0;

    return 1;
}
```

Listing 15. Four-way safety check for obstacle placement.

D. Color Themes

Four themes (Classic, Ice, Lava, Rainbow) are stored as `static const` arrays of ANSI 256-colour codes. The renderer reads theme codes rather than hard-coded numbers, so adding a new theme requires only adding a row to the array. The Rainbow theme additionally cycles its body colour every segment via a separate look-up; this is enabled by a one-bit flag in the theme struct rather than a separate code path. Figure 4 compares the four themes in Classic mode.

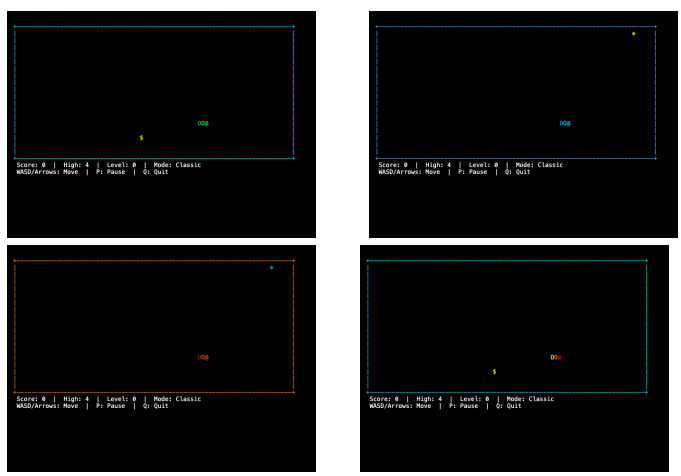

Fig. 4. All four themes. Top row: Classic (green), Ice (blue). Bottom row: Lava (red/orange), Rainbow (cycling). The HUD and border colours adapt with each theme.

E. Two Game Modes

The board exposes two collision policies, selected on the title screen:

- **Classic mode** – the wall is lethal; running into the border kills the snake.

- **Wrap-Around mode** – the board behaves as a torus; coordinates leaving any edge re-enter on the opposite edge via the branch-free `my_mod` expression of Listing 7 (see Fig. 5).

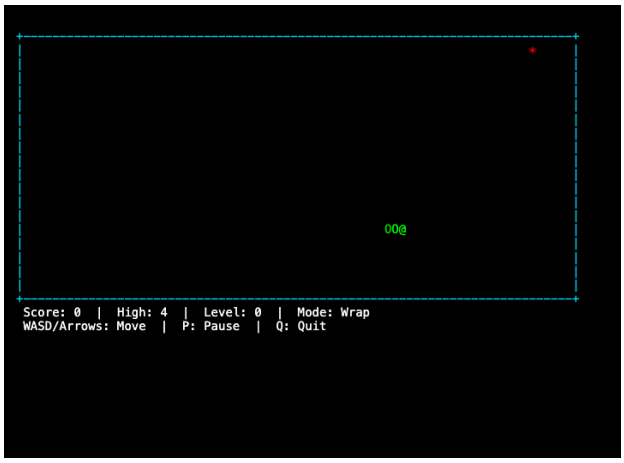

Fig. 5. Wrap-Around mode. The HUD reads `Mode: Wrap`; the snake can leave any wall and re-emerge on the opposite side.

VI. Phase 2 Developmental Features

In addition to the Phase 1 build documented above (branch `main` / `feature/endsem-submission`), a Phase 2 branch (`ravi/phase2`) extends the core game with several advanced mechanics. These extensions inherit the same custom libraries, allocator, and rendering pipeline; they exercise the same OS sub-systems but at a higher level of game complexity.

A. Multi-Food Spawning

The food module is generalised from a single `Food*` to a `Foods` container holding up to several pieces simultaneously. Each piece carries its own type and despawn timer; the collision and respawn logic of Section IV is iterated across the array. This raises the average tick speed without raising the difficulty floor, because the player has more targets to chase.

B. Ghost Snake (Autonomous AI Body)

A second snake (`Snake *ghost`) moves on the same board under autonomous control. Each tick `ghost_step()` chooses a direction with the same custom PRNG used for food and obstacle placement (Listing 11) and moves the ghost; its body is rendered in a distinct colour (Fig. 6, purple `oog`) and acts as a moving obstacle. The collision pipeline is extended so that landing on any ghost segment is lethal in Classic mode.

C. Power-Up Zones

Discrete power-up zones are spawned with a cooldown and a fixed lifetime. Two types are implemented:

- `POW_BOOST` – temporarily speeds the snake for 25 ticks.
- `POW_SLOW` – temporarily slows the snake for 25 ticks.

Zones glow with an animated colour cycle driven by a per-zone `glow_phase` counter, and they participate in obstacle/food/snake placement-safety checks via `powerups_find_at()`.

D. Shrinking Board (Inset)

The play area shrinks over time via a configurable inset. The board module exposes `board_in_play_area()` which all collision and spawn checks consult, so the shrink mechanism is enforced without modifying every caller. As the inset grows, food and power-ups outside the new play area are dropped through `powerups_drop_invalid()`.

E. Replay (Best-Run Ghost)

A persistent replay system records the head position of every tick into a `Replay` struct (capacity 6,000 frames; each frame is a pair of `short` coordinates = 4 bytes, so ≈24 KB total). When a run scores higher than the saved best, the trajectory is written to disk; subsequent runs load the best replay and render it as a translucent "ghost" that the player can race against. This pattern is the same one used by deterministic-replay debuggers in real OS schedulers.

F. Dynamic Terminal Resize Handling

Phase 2 honours `SIGWINCH`-style window changes: `wait_for_valid_terminal()` polls the terminal dimensions and pauses the game until the window meets the minimum size, after which `apply_terminal_resize()` re-clamps board geometry, drops out-of-bounds entities, and triggers a full redraw via `redraw_full_game()`. Figure 7 shows the Phase 2 build running across all four themes and both collision modes; every screenshot was generated by the same scripted pipeline that produced the Phase 1 figures.

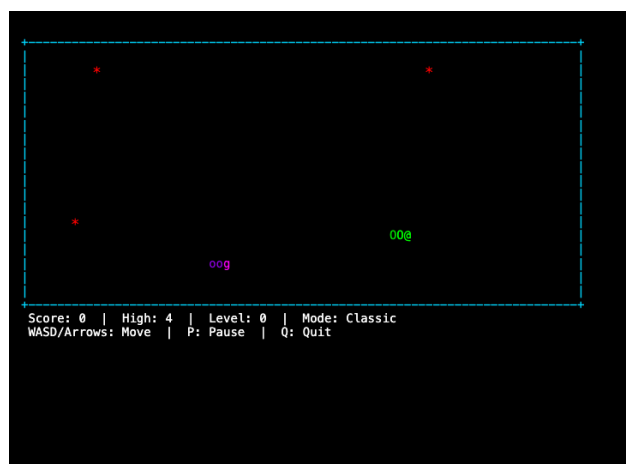

Fig. 6. Phase 2 build in action. Three red `*` food pieces are spawned simultaneously (multi-food); the purple `oog` body in the lower-middle is the autonomous ghost snake; the player snake `OO@` is in green at the right.

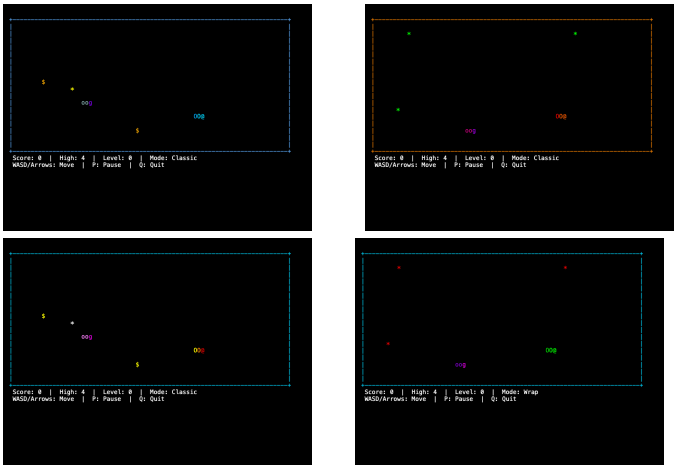

Fig. 7. Phase 2 across themes and modes. Top row: Ice and Lava themes with multi-food + ghost snake. Bottom row: Rainbow theme; Wrap-Around mode with the toroidal collision policy of Listing 7.

VII. Results and Testing

A. Build

The project builds cleanly under `gcc` with `-Wall -Wextra -Werror` on macOS Darwin 25.4.0 and Linux. A `Makefile` provides six phony targets: `all`, `run`, `test`, `clean`, `screenshots` (drives the binary in a real Terminal.app window via `osascript`+`screencapture`), and `screenshots-headless` (drives the binary inside a `tmux` pseudo-terminal and converts the captured ANSI to PNG via `aha` and headless Chromium). Total binary size is approximately 91 KB.

B. Reproducible Screenshot Pipeline

The `scripts/capture_screenshots.sh` driver scripts a complete play-through: it launches the binary inside a 80×24 `tmux` session, uses `tmux send-keys` to inject mode/theme selections and arrow-key inputs at a fixed cadence, snapshots the pane after each milestone with `tmux capture-pane -e`, converts the captured ANSI sequence to a self-contained HTML document via `aha`, and finally renders the HTML to PNG using a headless Chromium browser. A companion script `capture_phase2.sh` adds a `git worktree` on the `ravi/phase2` branch, builds the Phase 2 binary, and reuses the same driver to produce phase-2 screenshots without disturbing the `main` checkout. This pipeline guarantees that every figure in this report can be regenerated identically by typing `make screenshots-headless`.

C. Unit Tests

Three test suites exercise the system libraries:

- **math** – 32 cases covering multiply, divide, mod, abs, min, max, clamp, and PRNG period.
- **string** – 19 cases covering `strlen`, `strcpy`, `strcmp`, `strcat`, and integer/string conversions.
- **memory** – 14 cases covering allocation, deallocation, coalescing, fragmentation, and out-of-memory behaviour.

All 65 tests pass under `make test`.

D. Functional Demonstration

Figures 8–12 show representative screenshots from a play session.

Fig. 8. Title screen with the gradient ASCII “SNAKE” banner and game-mode selection (Classic / Wrap-Around).

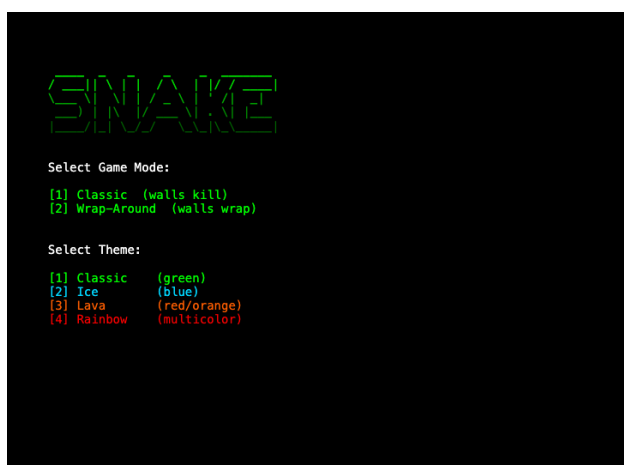

Fig. 9. Theme-selection screen offering Classic, Ice, Lava, and Rainbow palettes; each entry is rendered in its own colour as a live preview.

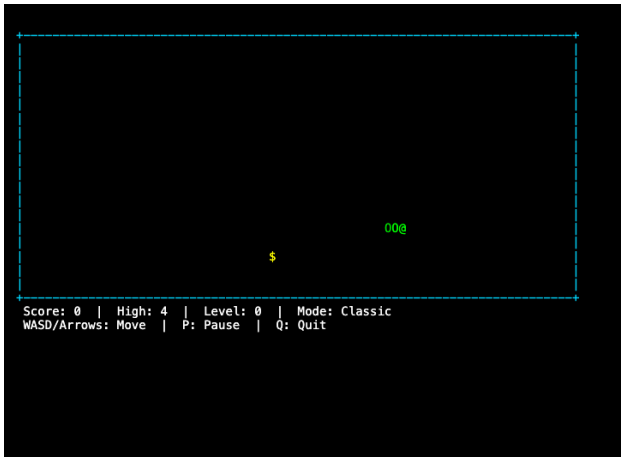

Fig. 10. In-game state in Classic mode with the Classic theme: green gradient snake (`OO@`), red food (`*`), cyan dashed border, and the two-line HUD showing score, high-score, level, mode, and key bindings.

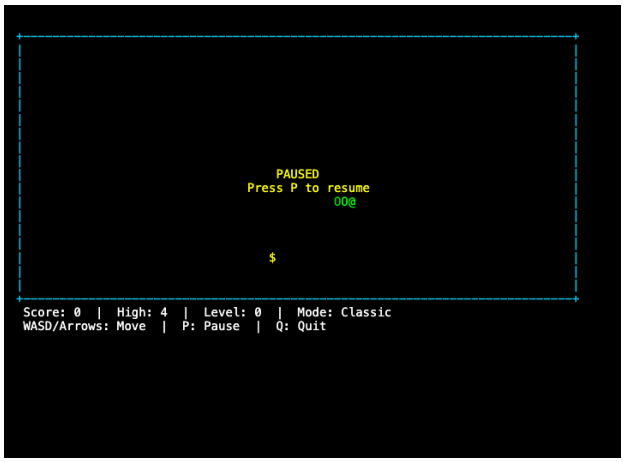

Fig. 11. Pause overlay (after pressing `P`). The game state is preserved exactly; the loop only polls the keyboard.

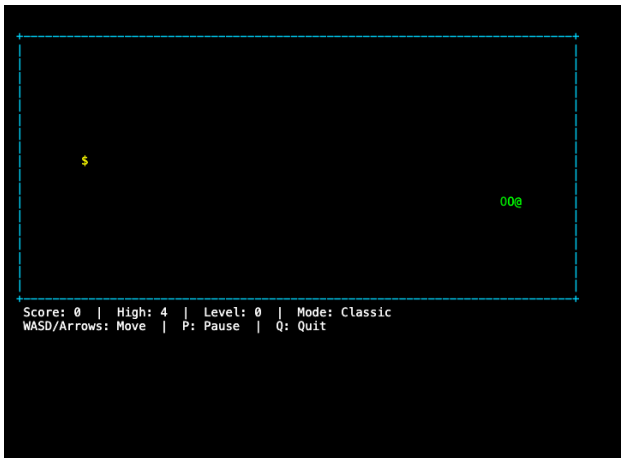

Fig. 12. Game-over screen showing the death animation, final and high score, and the lifetime statistics panel (games played, total food, longest snake, best level) – all values are persisted to `data/stats.dat`.

E. Version Control

The project is hosted publicly on GitHub at https://github.com/HackHeroic/Snake_game_os. The repository organises work across multiple branches:

- `main` / `feature/endsem-submission` – the stable Phase 1 submission documented in Sections IV–V.
- `ravi/phase2` – the Phase 2 extensions (multi-food, ghost snake, power-ups, shrinking board, replay) documented in Section VI.
- `phase2/features` – exploratory work on individual phase 2 features.

Significant milestones include:

- `feat: snake game core` – baseline gameplay, custom libraries.
- `feat: special food types` – bonus and slow food.
- `feat: obstacles, themes, stats` – level-up obstacles, four colour themes, persisted lifetime statistics.
- `feat: multi food logic` – generalised `Food*` to `Foods` container.
- `feat: ghost snake` – autonomous AI body sharing the board.
- `feat: phase 2 features` – power-up zones, shrinking board, replay ghost.
- `fix: dynamic terminal resize` – `SIGWINCH`-aware redraw and clamp.
- `fix: resize ghosting` – residual cells purged on inset shrink.
- `fix: obstacles_init before food_spawn` – avoids reading uninitialised memory.
- `docs: implementation document, viva docs, README updates.`

A linear, well-described commit history serves as both an evaluation artefact and a reproducibility guarantee.

VIII. Future Scope

The current system covers all five OS sub-systems but several interesting extensions remain.

a) (a) Multi-threaded Rendering and Input: Splitting the game loop into a renderer thread and an input thread would decouple frame timing from input latency. Synchronisation would require introducing a small POSIX-thread mutex layer, providing a teaching opportunity for race conditions and deadlock.

b) (b) Free-Block Lists for the Allocator: The current first-fit traversal is $O(n)$ in the number of blocks. Maintaining an explicit free-list (and, optionally, segregated free-lists by size class) would reduce average allocation time at the cost of additional bookkeeping.

c) (c) Networked Multiplayer: A simple TCP server in front of two game instances would demonstrate I/O multiplexing via `select` or `epoll`. The existing strict separation of `game/` from `ui/` makes this conceptually straightforward.

d) (d) Crash-Safe File System: Replacing the current rewrite-in-place save with a copy-on-write rename pattern (write to `stats.dat.tmp`, `rename` into place) would survive a crash mid-write, mirroring the journaling approach used by ext4 and APFS [1].

e) (e) Network-Synchronised Ghost Snakes: Phase 2 already implements a local ghost snake (Section VI); a natural next step is to share ghost trajectories between users over a small UDP protocol so that two players can race each other's best runs asynchronously.

f) (f) Sandboxed Resource Limits: Setting `rlimit` bounds on CPU and memory at start-up would demonstrate kernel-enforced security, complementing the application-level invariants already in place.

g) (g) Replay-File Format Stability: The Phase 2 replay system writes a fixed-size binary record. A versioned, length-prefixed format (with a magic number and forward-compatible reserved fields) would let future builds read older replays, much as ext4 reads ext2 superblocks.

h) (h) Audio via Coreaudio / ALSA: A single-channel beep on food intake and game-over would round out the I/O subsystem and demonstrate device management beyond the terminal.

IX. Conclusion

We presented *Snake Game OS*, a real-time terminal Snake game in which all five canonical operating-systems sub-systems—memory, processes, files, I/O, and security—are implemented from first principles in approximately 1,500 lines of C. The design choices are deliberate and defensible: a first-fit allocator with coalescing matches the small, homogeneous workload; a deterministic game-loop scheduler with a three-state FSM models a cooperative process; a delta-render strategy over ANSI escape codes eliminates flicker; flat ASCII files persist user data; and a coordinated halt protocol guarantees terminal restoration on every exit path. A separate Phase 2 branch lifts the engine to a higher complexity tier with multi-food, an autonomous ghost AI, power-up zones, a shrinking board, persistent best-run replay, and `SIGWINCH`-aware resize handling – all built atop the same five sub-systems without bypassing the custom allocator or the strict header allow-list. The 65 unit tests, the strict `-Werror` build, the scripted screenshot pipeline, and the public GitHub history together form an evaluation surface that is reproducible, inspectable, and pedagogically transparent.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson, 2014.

[3] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Reading, MA: Addison-Wesley, 1997.

[4] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic storage allocation: A survey and critical review," in *Proc. Int. Workshop Memory Mgmt.*, Sept. 1995, pp. 1–116.

[5] S. K. Park and K. W. Miller, "Random number generators: Good ones are hard to find," *Commun. ACM*, vol. 31, no. 10, pp. 1192–1201, Oct. 1988.

[6] L. Schrage, "A more portable Fortran random number generator," *ACM Trans. Math. Softw.*, vol. 5, no. 2, pp. 132–138, June 1979.

[7] T. E. Dickey, *NCURSES—New Curses*, GNU project documentation, 2024. [Online]. Available: https://invisible-island.net/ncurses/

[8] *Information technology—Control functions for coded character sets*, ECMA Standard 48, 5th ed., June 1991.

[9] IEEE Std 1003.1-2017 (Revision of IEEE Std 1003.1-2008), *IEEE Standard for Information Technology—Portable Operating System Interface (POSIX)*, IEEE, 2018.